

Course Dashboard - Erarbeitung / Reflexion

ZIEL DER UNTERSUCHUNG

Ausgangspunkt für die Umsetzung des Course Dashboards ist das in Phase 1 entwickelte UML-Klassendiagramm. In dieser Phase soll untersucht werden, wie sich die dort modellierten objektorientierten Konzepte mit Python umsetzen lassen und ob sich aus den Besonderheiten von Python Änderungen am bisherigen Modell ergeben.

Dabei werden insbesondere Klassen und Objekte, Properties, abgeleitete Eigenschaften, Komposition, Enums und Vererbung untersucht. Zusätzlich wird betrachtet, wie die fachlichen Klassen später mit der grafischen Benutzeroberfläche und der Datenspeicherung zusammenarbeiten können.

Die Untersuchung soll nicht nur die technische Umsetzbarkeit überprüfen, sondern auch dazu dienen, das UML-Modell aus Phase 1 hinsichtlich der späteren Python-Implementierung zu überprüfen und gegebenenfalls anzupassen.

OBJEKTORIENTIERTE KONZEPTE IN PYTHON

Klassen und Konstruktoren

Als Beispiel wird eine Klasse "ExamResult" definiert mit den beiden Werten "name" und "grade".

```
class ExamResult:
    def __init__(self, name, grade):
        self.name = name
        self.grade = grade

exam = ExamResult("Python Project Report", 1.2)

print(f'Exam Result: {exam.name}, Grade: {exam.grade:1}')
```

```
Exam Result: Python Project Report, Grade: 1.2
```

Ergebnis

Python-Klassen lassen sich grundsätzlich direkt aus dem bisherigen UML-Modell ableiten. Die Konstruktion von Objekten erfolgt über die Methode `__init__()`.

Für das Course Dashboard werden Type Hints verwendet. Sie sind zwar nicht zwingend erforderlich, erleichtern aber insbesondere bei mehreren miteinander verbundenen Klassen die Lesbarkeit des Codes und machen erwartete Datentypen besser nachvollziehbar.

```
class ExamResult:
    def __init__(self, name: str, grade: float):
        self.name = name
        self.grade = grade
```

Das ursprüngliche UML-Modell kann in diesem Bereich daher grundsätzlich beibehalten werden.

Properties

Statt die Attribute einer Klasse öffentlich zugänglich zu machen, soll in diesem Projekt der Zugriff auf Werte durch Properties gewährt werden. Dadurch kann einerseits der Zugriff "von außen" auf eine Klasse gesichert werden, indem kein Änderungszugriff gewährt wird (kein Setter verfügbar ist). Andererseits können vollkommen neue Zugriffe gewährt werden, wie im darauffolgenden Beispiel "Abgeleitete Eigenschaften".

```
class Module:
    def __init__(self, name):
        self._name = name
```

```

@property
def name(self):
    return self._name

@name.setter
def name(self, value: str):
    self._name = value

module = Module("Python Dashboard Project")
print(f'A new Module started: {module.name}')

module.name = "Python Dashboard Project 2000"
print(f'Module was renamed: {module.name}')

```

```

A new Module started: Python Dashboard Project
Module was renamed: Python Dashboard Project 2000

```

Ergebnis

Python ermöglicht sowohl den direkten Zugriff auf Attribute als auch die Kapselung über `@property`. Für das Projekt werden Properties dort eingesetzt, wo ein kontrollierter Zugriff oder eine Berechnung sinnvoll ist.

Ein wesentlicher Unterschied zu einer klassischen Implementierung mit privaten Feldern besteht darin, dass Python keine strikt privaten Attribute im gleichen Sinne wie beispielsweise C# erzwingt. Die Schreibweise `_name` kennzeichnet ein Attribut in Python primär als internes Attribut und stellt keine harte Zugriffssperre dar.

Properties werden daher im Projekt nicht für jedes einzelne Attribut automatisch verwendet, sondern gezielt dort, wo dadurch ein fachlicher Mehrwert entsteht. Insbesondere abgeleitete Werte wie `isPassed` oder `isCompleted` eignen sich für Properties.

Dadurch ergibt sich eine Anpassung gegenüber dem ursprünglichen UML-Modell: Werte, die aus anderen Daten berechnet werden können, müssen nicht als eigenständige gespeicherte Attribute modelliert werden.

Abgeleitete Eigenschaften

Durch die Untersuchung wurde festgestellt, dass es im Datenmodell mehrere Werte gibt, die nicht unabhängig gespeichert werden sollten:

- ExamResult: passed → wurde mit dem Prüfungsergebnis das Prüfungsziel erreicht?
- Module: completed → wurde das Modul abgeschlossen? also alle ExamResults "passed"?
- Studienfortschritt
- Notendurchschnitt

```

class ExamResult:
    def __init__(self, grade):
        self._grade = grade

    @property
    def grade(self):
        return self._grade

    @property
    def passed(self):
        return self._grade <= 4.0

exam = ExamResult(1.3)
print(f'Exam Result: {exam.grade} ... passed: {exam.passed}')

```

```

Exam Result: 1.3 ... passed: True

```

Ergebnis

Anstatt diese Werte direkt zu speichern, werden Properties definiert, die jeweils das aktuelle Ergebnis berechnen und zurückgeben. Das verhindert widersprüchliche Zustände. Wird beispielsweise eine Note geändert, müssen "passed", "completed"

und der Studienfortschritt nicht separat aktualisiert werden. Bei einer Abfrage werden diese Werte aus dem aktuellen Datenbestand ermittelt.

Komposition

Bei der Untersuchung wurde festgestellt, dass die im UML-Diagramm dargestellten Kompositionen in Python durch Objektverweise beziehungsweise Listen umgesetzt werden können.

```
class ExamResult:
    def __init__(self, grade):
        self.grade = grade

class Module:
    def __init__(self, name):
        self.name = name
        self.exam_results = []

class Semester:
    def __init__(self, number):
        self.number = number
        self.modules = []

class DegreeProgram:
    def __init__(self, name):
        self.name = name
        self.semesters = []

program = DegreeProgram("SQL & Python Programming")

semester = Semester(1)
module = Module("SQL Data Mart Project")

semester.modules.append(module)
program.semesters.append(semester)
```

Ergebnis

Die Komposition aus dem UML-Modell lässt sich in Python durch Objekte, die andere Objekte beziehungsweise Listen von Objekten enthalten, umsetzen.

Dabei ist insbesondere der Lebenszyklus der Objekte relevant. Ein "Module" wird innerhalb des betrachteten "Semester" verwaltet und ein "ExamResult" innerhalb eines "Module". Werden diese Objekte aus dem übergeordneten Objekt entfernt und bestehen keine weiteren Referenzen darauf, können sie anschließend auch nicht mehr über die jeweilige fachliche Struktur erreicht werden.

Die Listenattribute, die zur technischen Umsetzung der Komposition benötigt werden, werden daher nicht als eigenständige Attribute im fachlichen UML-Modell dargestellt. Sie gehören zur späteren Implementierung der Klassen.

Enums

Enums lassen sich mittels der enum-Bibliothek in Python sehr einfach umsetzen, und sind ideal für die Anforderungen an das "ExamResult" bzw. den "ExamType".

```
from enum import Enum

class ExamType(Enum):
    ONLINE_TEST = "Online Test"
    PROJECT = "Project"
    MANUSCRIPT = "Manuscript"

class ExamResult:
    def __init__(self, exam_type, grade):
        self.exam_type = exam_type
        self.grade = grade

exam = ExamResult(ExamType.PROJECT, 1.7)
```

```
print(f'Exam Type: {exam.exam_type}, Grade: {exam.grade}')
```

```
Exam Type: ExamType.PROJECT, Grade: 1.7
```

Vererbung

Technisch wäre es möglich, die drei verschiedenen Typen als Vererbung einer Basisklasse darzustellen. Ich habe mich jedoch dagegen entschieden, da es den Prototypen unnötig verkompliziert und keinerlei sinnvolle Funktion hinzufügt.

Ein kurzes Beispiel einer Vererbung: "ProjectResult" beerbt die Basisklasse "ExamResult".

```
class ExamResult:
    def __init__(self, grade):
        self.grade = grade

class ProjectResult(ExamResult):
    def __init__(self, grade, project_name):
        super().__init__(grade)
        self.project_name = project_name
```

ÄNDERUNGEN AM UML-MODELL

Die Untersuchung der objektorientierten Konzepte in Python hat gezeigt, dass das grundlegende fachliche Modell aus Phase 1 weiterhin geeignet ist. Einige Elemente werden jedoch angepasst, um die fachliche Modellierung klarer von der späteren technischen Implementierung zu trennen.

Entfernen der Listenattribute

Die Attribute "semesters", "modules" und "examResults" werden aus dem fachlichen UML-Modell entfernt. Die Beziehungen zwischen den Klassen werden bereits durch die Kompositionen dargestellt. Die entsprechenden Listen werden erst bei der Python-Implementierung benötigt.

Entfernen von getSemester() und getModule()

Die Methoden getSemester() und getModule() werden aus dem fachlichen UML-Modell entfernt, da es sich hierbei hauptsächlich um Zugriffsmethoden handelt. Die konkrete Umsetzung des Zugriffs erfolgt später über Python-Attribute beziehungsweise Properties.

Änderung von moduleNumber zu code

Das Attribut "moduleNumber" wird in "code" umbenannt. Die Bezeichnung code ist eindeutiger und berücksichtigt, dass die Kennung nicht zwingend ausschließlich aus numerischen Zeichen bestehen muss.

Abgeleitete Eigenschaft isPassed

Der Bestehensstatus eines "ExamResult" wird nicht separat gespeichert. Stattdessen wird "isPassed" (umbenanntes Property: "passed") aus der erreichten Note abgeleitet.

Abgeleitete Eigenschaft isCompleted

Auch der Abschlussstatus eines "Module" wird nicht unabhängig gespeichert, sondern über ein abgeleitetes Property "isCompleted" (umbenanntes Property: "completed"). Ein Modul gilt als abgeschlossen, wenn die dafür erforderlichen Prüfungsleistungen bestanden wurden.

Studienfortschritt und Notendurchschnitt

Der Studienfortschritt und der aktuelle Notendurchschnitt werden als berechnete Werte betrachtet. Dadurch werden diese Werte nicht redundant im Datenmodell gespeichert.

Verzicht auf Vererbung

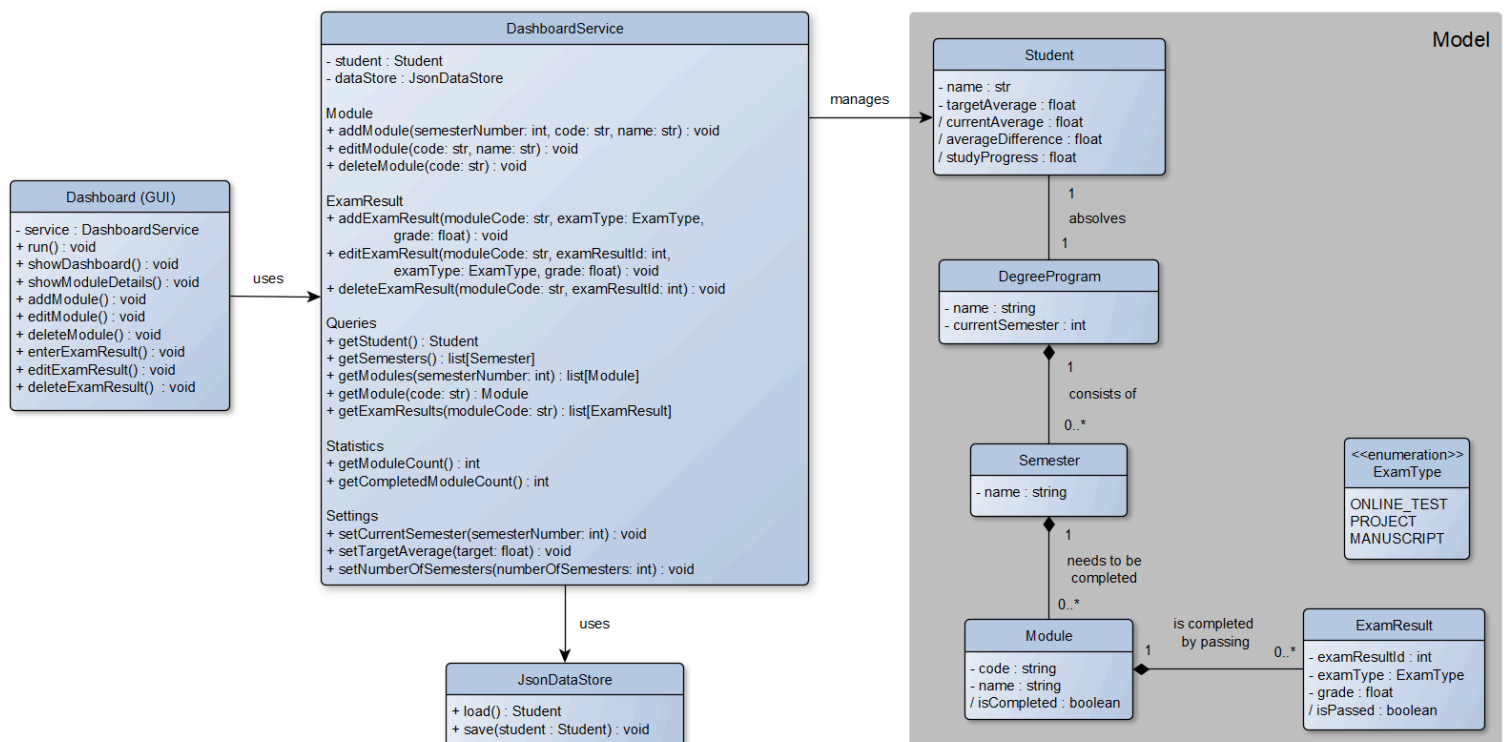
Die Untersuchung hat bestätigt, dass Python die benötigte Vererbung technisch problemlos unterstützt. Für das Course Dashboard wird dennoch keine Vererbung für die verschiedenen Prüfungsarten eingesetzt. ExamType bildet diese Unterscheidung ausreichend ab und hält das Modell übersichtlich.

Ergebnis

Das fachliche UML-Modell wird somit nicht grundlegend verändert, sondern an mehreren Stellen vereinfacht und von technischen Implementierungsdetails bereinigt.

GESAMTARCHITEKTUR

UML-Diagramm



Übersicht der benötigten Klassen

Dashboard	Verantwortlich für die Darstellung und Benutzerinteraktion.
DashboardService	Kapselt die Anwendungslogik und vermittelt zwischen GUI und fachlichem Modell.
JsonDataStore	Verantwortlich für das Laden und Speichern der Daten. Dadurch wird die Speicherung vom fachlichen Modell getrennt.
Student	Informationen über den Studenten, Notenziel
DegreeProgram	Studiengang und Studienfortschritt
Semester	Module eines Semesters

Module	Modul und dessen Prüfungsleistungen
ExamResult	einzelne Prüfungsleistung

Initialisierung der Anwendung

Die Erstellung und Verbindung der Objekte erfolgt in der Startdatei *main.py*. Dadurch werden die Abhängigkeiten der einzelnen Komponenten an einer zentralen Stelle festgelegt.

Der Ablauf beim Programmstart ist:

1. *main.py* erstellt einen *JsonDataStore*.
2. Der "JsonDataStore" lädt die gespeicherten Daten aus der JSON-Datei.
3. Aus den geladenen Daten wird das Domain Model mit "Student", "DegreeProgram", "Semester", "Module" und "ExamResult" aufgebaut.
4. *main.py* erstellt den "DashboardService" und übergibt ihm das geladene "Student"-Objekt sowie den "JsonDataStore".
5. Anschließend wird das "Dashboard" erstellt und erhält den "DashboardService".
6. Das "Dashboard" erstellt die Tkinter-Benutzeroberfläche und verbindet die UI-Elemente mit den entsprechenden Event-Handlern.
7. Mit *dashboard.run()* wird die Tkinter Event Loop gestartet.

Die GUI greift damit nicht direkt auf die JSON-Datei zu. Änderungen werden über den "DashboardService" durchgeführt, der anschließend den "JsonDataStore" zum Speichern verwendet.

Beispiel: Prüfungsleistung hinzufügen

Wenn der Benutzer über die GUI eine neue Prüfungsleistung eingibt, wird zunächst der entsprechende Event-Handler des "Dashboard" ausgelöst.

Das "Dashboard" liest die Eingaben des Benutzers aus der Benutzeroberfläche und ruft anschließend *DashboardService.addExamResult()* auf. Der Service sucht das entsprechende Modul, erstellt ein neues "ExamResult" und fügt es dem Modul hinzu.

Anschließend wird der aktualisierte Datenbestand über *JsonDataStore.save()* in der JSON-Datei gespeichert.

Der Bestehensstatus muss dabei nicht separat aktualisiert werden. Er ergibt sich automatisch aus der gespeicherten Note. Ebenso können der aktuelle Notendurchschnitt und der Studienfortschritt bei der nächsten Darstellung des Dashboards aus den aktuellen Daten berechnet werden.

Anschließend aktualisiert das Dashboard die angezeigten Werte.

Begründung der Architektur

Die Architektur trennt die drei wesentlichen Verantwortungsbereiche des Prototyps voneinander.

Das "Dashboard" ist ausschließlich für die Benutzeroberfläche und die Verarbeitung von Benutzerinteraktionen zuständig. Die eigentliche Anwendungslogik wird im "DashboardService" gebündelt. Dadurch muss die GUI beispielsweise nicht wissen, wie ein Modul innerhalb eines Semesters gespeichert oder wie die JSON-Datei aufgebaut ist.

Der "JsonDataStore" kapselt die Persistenz. Dadurch sind die fachlichen Klassen unabhängig davon, ob die Daten später in einer JSON-Datei, einer Datenbank oder einer anderen Speicherform abgelegt werden.

Die fachlichen Klassen bilden weiterhin die Struktur des Studiengangs und die darin enthaltenen Prüfungsleistungen ab. Fachliche

Werte wie "isPassed", "isCompleted", der Notendurchschnitt und der Studienfortschritt werden möglichst aus den vorhandenen Daten abgeleitet, anstatt redundant gespeichert zu werden.

FAZIT

Die Untersuchung hat gezeigt, dass sich das fachliche Modell grundsätzlich mit Python umsetzen lässt. Einige Elemente des ursprünglichen UML-Modells wurden jedoch angepasst, um fachliche Konzepte und technische Implementierungsdetails klarer voneinander zu trennen. Insbesondere Properties und abgeleitete Eigenschaften ermöglichen es, redundante Zustände zu vermeiden. Für die technische Umsetzung wurden mit "Dashboard", "DashboardService" und "JsonDataStore" drei zusätzliche Klassen eingeführt. Dadurch werden Benutzeroberfläche, Anwendungslogik und Persistenz getrennt, ohne den Prototypen durch eine unnötig komplexe Architektur zu überladen.